Lecture 11: DiT, Consistency Models, and Visual Autoregressive Models

최신 아키텍처와 가속 샘플링 — Transformer, 일관성 모델, 스케일별 자기회귀

목차 (Table of Contents)

- 0. 개요
  - 선수 지식
  - 학습 목표
- Part A: DiT (Diffusion Transformer)
  - A.1 동기
  - A.2 구조
  - A.3 adaLN-Zero — 핵심 조건 주입 메커니즘
  - A.4 모델 변형과 스케일링
  - A.5 U-Net vs DiT
  - A.6 MM-DiT (Stable Diffusion 3)
  - A.7 PyTorch 구현: DiT Block (adaLN-Zero)
  - A.8 PyTorch 구현: Patchify / Unpatchify
- Part B: Consistency Models — 1~4 스텝 생성
  - B.1 동기
  - B.2 핵심 아이디어: 자기 일관성 (Self-Consistency)
  - B.3 경계 조건
  - B.4 Consistency Distillation
  - B.5 Consistency Training (증류 없이)
  - B.6 생성
  - B.7 성능
  - B.8 PyTorch 구현: Consistency Model 추론
- Part C: Visual Autoregressive Models (VAR)
  - C.1 기존 Visual AR의 한계 (강의 5 DALL-E)
  - C.2 VAR의 핵심: Next-Scale Prediction
  - C.3 구조
  - C.4 기존 토큰별 AR vs VAR 비교
  - C.5 스케일링 법칙의 발견
  - C.6 MAR: VQ 없는 Visual AR
  - C.7 PyTorch 구현: VAR Next-Scale Prediction (간소화)
- Part D: 하이브리드 모델 — AR + Flow Matching
  - D.1 FlowAR (2024)
  - D.2 패러다임 수렴

- 핵심 수식 요약
- 다음 강의 미리보기
- 참고문헌 (References)

---

0. 개요

이 강의는 2023~2025년의 세 가지 핵심 발전을 다룬다:

주제	핵심 질문	대표 논문
DiT	U-Net 대신 Transformer를 쓰면?	📄 Peebles & Xie, ICCV 2023
Consistency Models	1~4 스텝으로 생성할 수 없을까?	📄 Song et al., ICML 2023
VAR	이미지도 자기회귀로 스케일링 할 수 있을까?	📄 Tian et al., NeurIPS 2024 Best Paper

> 💡 **왜 이 세 가지가 중요한가?** 2020~2022년의 생성 모델은 두 가지 근본적 병목에 직면했다. 첫째, **아키텍처 병목**: U-Net은 CNN 기반이라 모델을 키워도 성능 향상이 예측 불가능했다. LLM에서는 "파라미터 2배 → 성능 X만큼 개선"이라는 스케일링 법칙(Scaling Law)이 성립하는데, 이미지 생성에서는 이런 법칙이 없었다. 둘째, **속도 병목**: Diffusion 모델은 고품질 이미지 하나를 생성하는 데 수십~수천 번의 신경망 호출이 필요했다. 이 세 논문은 각각 아키텍처(DiT), 속도(Consistency), 그리고 패러다임 자체(VAR)를 혁신하여 이 병목들을 돌파한다.

선수 지식

이 강의를 이해하기 위해 다음 개념에 대한 사전 지식이 필요하다:

선수 지식	관련 강의	핵심 개념
DDPM / Score-based Models	강의 6~7	노이즈 스케줄, ϵ-prediction, 역방향 ODE/SDE
Flow Matching	강의 9~10	ODE 궤적, 확률 경로, 벡터장 회귀
Transformer	강의 3	Multi-Head Self-Attention, LayerNorm, Positional Embedding
VQ-VAE / VQGAN	강의 4~5	벡터 양자화, 코드북, 이산 토큰 표현

선수 지식 보충 설명:

- **ODE 궤적 (ODE Trajectory)**: Diffusion 모델에서 데이터 x_0에 노이즈를 점진적으로 추가하면 $x_0 \to x_{t_1} \to \cdots \to x_T$라는 경로가 만들어진다. 이 경로는 ODE(Ordinary Differential Equation, 상미분방정식) $dx = v(x, t)\, dt$의 해(solution)로 표현할 수 있으며, 이 해의 궤적을 "ODE 궤적"이라 부른다. 역방향으로 이 ODE를 풀면 노이즈에서 데이터를 복원할 수 있다. Consistency Model은 이 궤적의 성질을 직접 활용한다.
- **Layer Normalization**: 신경망의 각 레이어에서 hidden state h의 평균과 분산을 정규화하여 $\hat{h} = (h - \mu)/\sigma$로 만드는 기법이다. 학습을 안정화하고 수렴 속도를 높인다. 일반적인 LayerNorm은 정규화 후 학습 가능한 scale(γ)과 shift(β)를 적용하는데, DiT의 adaLN은 이 γ, β를 고정 파라미터가 아닌 **조건 벡터로부터 동적으로 생성**한다.
- **벡터 양자화 (Vector Quantization)**: 연속 벡터를 유한한 코드북(codebook)의 가장 가까운 벡터로 매핑하는 기법이다. VQ-VAE에서는 인코더 출력을 코드북 벡터로 양자화하여 이미지를 이산 토큰의 시퀀스로 표현한다. 이렇게 하면 이미지를 언어 모델처럼 다룰 수 있게 된다.

학습 목표

이 강의를 마치면 다음을 할 수 있어야 한다:

1. DiT가 U-Net 기반 디퓨전 모델을 어떻게 대체하는지 설명할 수 있다 (adaLN-Zero 조건 주입 포함)
2. Consistency Model의 자기 일관성 속성을 이해하고, 1-step 생성이 왜 가능한지 수학적으로 설명할 수 있다
3. VAR의 next-scale prediction이 기존 next-token prediction과 어떻게 다르며, 왜 스케일링 법칙이 나타나는지 설명할 수 있다
4. 세 모델 각각의 PyTorch 핵심 모듈을 구현할 수 있다
5. AR과 Diffusion의 융합 트렌드를 파악하고 향후 방향을 전망할 수 있다

Part A: DiT (Diffusion Transformer)

> 📄 Peebles & Xie, "Scalable Diffusion Models with Transformers", ICCV 2023

A.1 동기

DDPM/LDM은 U-Net을 사용했다. 하지만 LLM에서 Transformer의 **스케일링 법칙**이 검증된 이후:

> "생성 모델에서도 Transformer를 쓰면 같은 스케일링 법칙이 성립할까?"

> 💡 **왜 U-Net을 Transformer로 대체해야 하는가?**
>
> U-Net은 2015년 의료 영상 분할을 위해 설계된 CNN 아키텍처다. Diffusion 모델에서 뛰어난 성능을 보였지만, 근본적인 한계가 있다:

> **1. Locality bias (지역성 편향)**: CNN의 합성곱 필터는 본질적으로 **국소적(local)**이다. 3x3 필터는 주변 9개 픽셀만 본다. 여러 레이어를 쌓으면 수용 영역(receptive field)이 넓어지지만, 이미지의 먼 영역 간 관계 (예: 사람의 양쪽 눈이 대칭이어야 한다, 배경의 조명 방향이 일관되어야 한다)를 포착하기 어렵다. 반면 Transformer의 Self-Attention은 **모든 위치 쌍**을 직접 비교하므로, 이미지의 어떤 두 영역이든 한 번의 레이어로 관계를 포착할 수 있다.
>
> **2. 스케일링 예측 불가능**: U-Net에서 모델을 키우려면 채널 수, 레이어 수, 해상도 단계 수 등 여러 하이퍼파라미터를 동시에 조정해야 한다. 어떤 조합이 최적인지 사전에 알기 어렵고, 컴퓨팅을 2배로 늘렸을 때 성능이 얼마나 개선될지 예측할 수 없다. 반면 Transformer는 GPT 시리즈에서 검증된 바와 같이, **파라미터 수 vs. 성능** 관계가 거듭제곱 법칙(power law)을 따른다: $L \propto N^{-\alpha}$ (Loss는 파라미터 수 N의 거듭제곱에 반비례). 이 법칙 덕분에 "GPU를 X배 더 쓰면 성능이 Y만큼 개선된다"는 예측이 가능해진다.
>
> **3. 실증적 증거 — FID vs GFlops**: DiT 논문의 Figure 1은 이 점을 명확히 보여준다. 모델의 연산량(GFlops)을 x축에, FID(생성 품질, 낮을수록 좋음)를 y축에 놓으면, DiT는 GFlops가 증가할수록 FID가 **일관되고 예측 가능하게** 감소한다. U-Net 기반 모델들은 이런 깔끔한 곡선을 보여주지 못한다. 이것이 DiT의 핵심 가치다: 더 많은 컴퓨팅을 투입하면 **보장된** 성능 향상을 얻을 수 있다.

A.2 구조

```
입력: 노이즈 잠재 표현 z_t (32×32×4, from VAE)
        ↓
┌──────────────────────────────────────┐
│ Patchify                             │
│  32×32×4 → 2×2 패치 → 256개 토큰       │
│  각 토큰: 2×2×4 = 16차원 → Linear → d  │
│        ↓                             │
│ + Positional Embedding (학습 가능)      │
│        ↓                             │
│ ┌──────────────────────────────┐     │
│ │        DiT Block × N         │     │
│ │                              │     │
│ │  [adaLN-Zero (조건: t + class)]  │
│ │        ↓                     │     │
│ │  [Multi-Head Self-Attention] │     │
│ │        ↓                     │     │
│ │  [adaLN-Zero]                │     │
│ │        ↓                     │     │
│ │  [MLP (Feed-Forward)]        │     │
│ │        ↓                     │     │
│ │  (반복 N번)                   │     │
│ └──────────────────────────────┘     │
│        ↓                             │
│ [adaLN → Linear → Reshape]           │
│        ↓                             │
│ Unpatchify: 256 토큰 → 32×32×(2×4)   │
│           노이즈 예측 + 분산 예측       │
└──────────────────────────────────────┘
```

Patchify 단계 상세 설명:

> 💡 **왜 이미지를 패치(patch)로 나누는가?** Transformer는 본래 **시퀀스(sequence)**를 처리하는 아키텍처다. 텍스트에서는 단어/토큰이 자연스러운 시퀀스 단위이지만, 이미지는 2D 격자(grid) 구조다. ViT(Vision Transformer)에서 제안된 해법은 이미지를 겹치지 않는 정사각형 패치로 분할하여 각 패치를 하나의 "토큰"으로 취급하는 것이다. 이렇게 하면 이미지를 Transformer가 처리할 수 있는 토큰 시퀀스로 변환할 수 있다.

Patchify의 구체적인 계산 과정을 단계별로 살펴보자:

1. **입력**: VAE 인코더가 출력한 잠재 표현 $z_t \in \mathbb{R}^{32\times32\times4}$ (높이 32, 너비 32, 채널 4)
2. **패치 분할**: 패치 크기 $p = 2$로 설정하면, 32×32 격자를 2×2 크기의 패치 $(32/2) \times (32/2) = 16 \times 16 = 256$개로 분할
3. **벡터화**: 각 패치는 $2 \times 2 \times 4 = 16$차원 벡터가 됨
4. **선형 투영**: 16차원 벡터를 Linear layer로 d차원(예: DiT-XL은 $d = 1152$)으로 투영
5. **위치 임베딩 추가**: 각 토큰에 학습 가능한 위치 임베딩(positional embedding)을 더하여 공간적 위치 정보를 부여

패치 크기의 트레이드오프: 패치 크기 p는 시퀀스 길이와 계산량을 직접 결정한다. $p = 2$이면 256개 토큰, $p = 4$이면 64개 토큰, $p = 8$이면 16개 토큰이 된다. Self-Attention의 계산 복잡도는 $O(T^2)$ (T: 토큰 수)이므로, 패치가 작을수록 토큰이 많아져 더 미세한 디테일을 포착할 수 있지만 계산 비용이 급격히 증가한다. DiT 논문에서는 $p = 2$가 FID 기준 최적임을 실험적으로 보여준다 (DiT-XL/2가 DiT-XL/4, DiT-XL/8보다 우수).

A.3 adaLN-Zero — 핵심 조건 주입 메커니즘

> 💡 **왜 조건 주입(conditioning) 방식이 중요한가?** Diffusion 모델은 "노이즈를 제거하여 이미지를 복원하는" 모델이다. 하지만 **어떤** 이미지로 복원할지는 조건(condition)이 결정한다. 예를 들어 "골든 리트리버" 클래스를 조건으로 주면 개 이미지가, "스포츠카"를 주면 자동차 이미지가 생성된다. 또한 타임스텝 t도 중요한 조건이다: t가 클 때(노이즈가 많을 때)는 전체적인 구조를 잡아야 하고, t가 작을 때(노이즈가 적을 때)는 세밀한 디테일을 다듬어야 한다. 따라서 조건을 모델에 **얼마나 효과적으로 주입하느냐**가 생성 품질을 좌우한다.

기존 Transformer의 LayerNorm을 **조건에 따라 변형**:

$$\mathrm{adaLN}(h, c) = \gamma(c) \odot \mathrm{LayerNorm}(h) + \beta(c)$$

기호	의미
h	현재 레이어의 hidden state
c	조건 벡터 ($= \mathrm{MLP}(t_{\mathrm{emb}} + y_{\mathrm{emb}})$, 시간+클래스)
$\gamma(c), \beta(c)$	c에서 MLP로 생성한 스케일/시프트

조건 주입 방식의 비교 — 왜 adaLN이 최선인가?

DiT 논문은 네 가지 조건 주입 방식을 실험적으로 비교한다:

1. **In-context conditioning (토큰 결합)**: 조건 벡터를 추가 토큰으로 시퀀스에 추가한다. 예: $[c, z_1, z_2, \ldots, z_{256}]$. 문제점: 조건 토큰은 시퀀스의 "다른 토큰 하나"일 뿐이며, Self-Attention에서 다른 토큰들이 이를 **무시할 수 있다**. 조건이 모델의 모든 레이어에 영향을 미친다는 보장이 없다.

2. **Cross-Attention**: 텍스트-이미지 모델(Stable Diffusion 1.x)에서 사용하는 방식. 별도의 Cross-Attention 레이어에서 이미지 토큰이 조건 토큰에 attend한다. 텍스트처럼 토큰 시퀀스인 조건에 적합하지만, 클래스 레이블이나 타임스텝 같은 **단일 벡터** 조건에는 과하다.

3. **Adaptive Layer Normalization (adaLN)**: 조건 벡터로부터 LayerNorm의 scale(γ)과 shift(β)를 동적으로 생성한다. **모든 레이어의 모든 토큰이 강제로 조건의 영향을 받는다** — LayerNorm은 Transformer 블록에서 피할 수 없는 연산이기 때문이다. 이것이 핵심 차이다.

4. **adaLN-Zero**: adaLN에 gate(α)를 추가하고 Zero 초기화한다. 아래에서 상세히 설명.

DiT 논문의 실험 결과, adaLN-Zero가 모든 방식 중 가장 낮은 FID를 달성했다.

"Zero": 학습 초기에 $\gamma = 1, \beta = 0, \alpha = 0$으로 시작:

$$\text{output} = h + \alpha(c) \odot \text{Attention}(\text{adaLN}(h, c))$$

$\alpha(c)$ 게이트가 0에서 시작 → 초기에는 각 블록이 항등 함수 → **안정적 학습 시작**

adaLN-Zero의 단계별 계산 과정:

조건 벡터 c로부터 출력까지의 전체 흐름을 추적해 보자:

```
Step 1: 조건 임베딩 생성
  t_emb = sinusoidal_embedding(t)      # 타임스텝 → 벡터 (강의 6 DDPM
  y_emb = class_embedding(y)           # 클래스 레이블 → 벡터
  c = MLP(t_emb + y_emb)              # 결합 조건 벡터 (B, D)

Step 2: 변조 파라미터 생성 (한 번에 6개)
  [γ₁, β₁, α₁, γ₂, β₂, α₂] = Linear(SiLU(c))   # (B, 6D) → chunk →

  여기서:
  γ₁, β₁, α₁ → Attention 경로용 (scale, shift, gate)
  γ₂, β₂, α₂ → MLP 경로용 (scale, shift, gate)

Step 3: Attention 경로
  h = LayerNorm(x)                     # 정규화 (affine=False: γ,β 없
  h = γ₁ ⊙ h + β₁                      # 조건부 scale & shift (adaLN
  h = MultiHeadSelfAttention(h)        # 표준 Self-Attention
  x = x + α₁ ⊙ h                       # gate 적용 후 잔차 연결

Step 4: MLP 경로 (동일 패턴)
  h = LayerNorm(x)
  h = γ₂ ⊙ h + β₂
  h = MLP(h)                           # Feed-Forward Network
  x = x + α₂ ⊙ h
```

> 💡 **왜 "Zero" 초기화가 핵심인가?** 학습 초기에 $\alpha = 0$이면, 위 Step 3에서 `x = x + 0 ⊙ h = x`가 된다. 즉, 각 DiT 블록이 입력을 그대로 출력하는 **항등 함수 (identity function)** 로 동작한다. 28개 블록을 쌓아도 초기에는 전체 네트워크가 입력을 그대로 통과시킬 뿐이다. 학습이 진행되면서 α가 서서히 0이 아닌 값을 갖게 되고, 각 블록이 점진적으로 기능을 수행하기 시작한다. 이것은 ResNet에서 residual branch를 zero-init하는 기법(Goyal et al., 2017)과 같은 원리다. 만약 α를 랜덤으로 초기화하면, 28개 블록의 랜덤한 변조가 누적되어 출력이 폭발하거나 소실될 수 있고, 학습 초반에 loss가 수렴하지 않는 불안정성이 발생한다.

> Cross-Attention(강의 8)과의 차이: Cross-Attention은 토큰별 세밀한 조건 주입(텍스트), adaLN은 글로벌 조건 주입(타임스텝, 클래스). Cross-Attention에서는 각 이미지 토큰이 텍스트의 서로 다른 부분에 attend할 수 있어서 "the **red** car on a **blue** road" 같은 토큰별 세밀한 조건 반영이 가능하다. 반면 adaLN은 **모든 토큰에 동일한** γ, β를 적용하므로, 타임스텝이나 클래스처럼 이미지 전체에 균일하게 영향을 미치는 조건에 적합하다.

⚠️ **adaLN-Zero의 초기화가 매우 중요하다**: α를 0이 아닌 값으로 초기화하면 학습 초반 불안정성이 급격히 증가하여 발산할 수 있다. 논문에서는 이 초기화 전략이 FID를 유의미하게 개선함을 실험적으로 보여준다.

A.4 모델 변형과 스케일링

모델	깊이 N	Hidden dim	Heads	파라미터	FID (ImageNet 256)
DiT-S/2	12	384	6	33M	68.4
DiT-B/2	12	768	12	130M	43.5
DiT-L/2	24	1024	16	458M	9.62
DiT-XL/2	28	1152	16	**675M**	**2.27**

/2: 패치 크기 2×2.

핵심 발견: 모델 크기가 커질수록 FID가 일관되게 개선. **LLM의 스케일링 법칙이 이미지 생성에서도 성립.**

스케일링 결과를 읽는 법:

위 표에서 주목할 점은 FID의 변화 패턴이다:

- S → B (파라미터 4배): FID 68.4 → 43.5 (36% 개선)
- B → L (파라미터 3.5배): FID 43.5 → 9.62 (78% 개선)
- L → XL (파라미터 1.5배): FID 9.62 → 2.27 (76% 개선)

파라미터 수를 늘릴 때마다 FID가 **단조 감소(monotonically decreasing)** 한다. 이것은 당연해 보일 수 있지만, U-Net 기반 모델에서는 이런 깔끔한 패턴이 나타나지 않았다. U-Net은 모델을 키울 때 채널 수, 레이어 수, Attention 해상도 등 여러 설계 선택이 얽혀 있어서 "최적의 스케일업 방법"을 찾기 어렵다. DiT는 ViT의 설계를 그대로 따르므로 (hidden dim과 depth만 조정), LLM의 스케일링 레시피를 직접 적용할 수 있다.

패치 크기에 따른 성능 비교:

모델	패치 크기	토큰 수	GFlops	FID
DiT-XL/2	2×2	256	119	**2.27**
DiT-XL/4	4×4	64	23	9.62

DiT-XL/8	8×8	16	6	36.7

패치 크기가 작을수록 (= 토큰 수가 많을수록) 성능이 좋지만, 계산량이 기하급수적으로 증가한다. /2 모델은 /8보다 약 20배 많은 연산을 사용하지만, FID는 16배 이상 개선된다. 이 트레이드오프를 이해하는 것이 실무에서 중요하다.

A.5 U-Net vs DiT

	U-Net (SD 1.x)	**DiT (SD 3, FLUX)**
구조	CNN + Attention 혼합	**순수 Transformer**
해상도 처리	다운/업샘플링	패치화 (고정 해상도)
스케일링	채널 수, 레이어 수 동시 조정	**파라미터 수만 늘리면 됨**
스킵 연결	✅ (Encoder→Decoder)	❌ (없어도 됨)
텍스트 조건	Cross-Attention	adaLN 또는 joint attention

각 차이점의 의미를 깊이 살펴보자:

- **해상도 처리**: U-Net은 인코더에서 해상도를 $256 \to 128 \to 64 \to 32$로 줄이고, 디코더에서 다시 $32 \to 64 \to 128 \to 256$으로 키운다. 이 과정에서 다운샘플링(정보 압축)과 업샘플링(정보 복원)이 정보 손실을 유발할 수 있다. DiT는 처음에 Patchify로 토큰을 만든 뒤 **고정된 시퀀스 길이**로 Transformer를 통과시킨다. 해상도 변화가 없으므로 정보 손실 경로가 없다.
- **스킵 연결**: U-Net의 스킵 연결은 인코더의 고해상도 특징을 디코더에 직접 전달하여 세밀한 디테일을 보존한다. DiT는 스킵 연결 없이도 Self-Attention이 모든 위치 간 정보를 직접 교환하므로 충분한 성능을 달성한다. 이는 아키텍처의 단순함이라는 큰 이점을 제공한다.
- **텍스트 조건**: U-Net 기반 Stable Diffusion 1.x/2.x는 Cross-Attention으로 텍스트를 주입한다. DiT 기반 모델들은 adaLN(글로벌 조건)과 joint attention(텍스트-이미지 합동 처리)을 결합하여, 조건 주입의 유연성과 효율성을 모두 확보한다.

A.6 MM-DiT (Stable Diffusion 3)

SD3의 아키텍처. 텍스트와 이미지를 **동등한 토큰**으로 처리:

> 💡 **왜 텍스트와 이미지를 "동등한 토큰"으로 처리하는가?** 기존 Stable Diffusion 1.x/2.x에서는 텍스트가 Cross-Attention을 통해 이미지에 "주입"되는 비대칭 구조였다. 이미지 토큰은 텍스트에 attend하지만, 텍스트 토큰은 이미지에 attend하지 않는다. 이는 텍스트가 이미지 생성을 **일방향으로만** 안내한다는 뜻이다. MM-DiT는 이 비대칭을 깨뜨려서, 텍스트 토큰도 이미지 토큰의 정보를 받고, 이미지 토큰도 텍스트 토큰의 정보를 받는 **양방향 정보 교환**을 가능하게 한다. 이를 통해 텍스트-이미지 정렬(alignment)이 크게 개선된다.

```
텍스트 토큰: [t₁, t₂, ..., t₇₇]
이미지 토큰: [z₁, z₂, ..., z₂₅₆]

결합: [t₁, ..., t₇₇, z₁, ..., z₂₅₆]
       ↓
[Joint Self-Attention]   ← 텍스트와 이미지가 서로 attend
       ↓
분리 → 각각의 MLP 처리
```

MM-DiT의 핵심 설계 세부사항:

- **Joint Self-Attention**: 텍스트 77개 + 이미지 256개 = 333개 토큰이 하나의 Self-Attention에 들어간다. 모든 토큰 쌍 간의 Attention weight가 계산되므로, 텍스트의 "red"라는 단어가 이미지의 자동차 영역 토큰과 강하게 연결되는 것이 가능해진다.
- **분리된 MLP**: Self-Attention 후에는 텍스트와 이미지 토큰을 다시 분리하여 **각각 독립적인 MLP**를 통과시킨다. 이는 텍스트와 이미지의 특징 공간(feature space)이 본질적으로 다르기 때문이다. Self-Attention에서는 정보를 교환하되, 특징 변환(feature transformation)은 각 모달리티에 맞게 별도로 수행하는 것이다.
- **adaLN과의 결합**: MM-DiT에서도 타임스텝 조건은 adaLN으로 주입한다. 텍스트 조건은 Joint Attention으로, 타임스텝 조건은 adaLN으로 — 각 조건의 특성에 맞는 주입 방식을 사용하는 셈이다.

A.7 PyTorch 구현: DiT Block (adaLN-Zero)

⚠️ 아래 코드는 DiT 논문의 핵심인 adaLN-Zero 블록을 PyTorch로 구현한 것이다. 조건 벡터 c로부터 scale(γ), shift(β), gate(α)를 생성하는 과정에 주목하라.

```python
import torch
import torch.nn as nn
import torch.nn.functional as F
import math


class AdaLNZeroDiTBlock(nn.Module):
    """
    DiT Block with adaLN-Zero conditioning.
    조건 벡터 c로부터 scale, shift, gate 파라미터를 생성하여
    LayerNorm을 적응적으로 변조하고, 출력에 gate를 적용한다.
    """
    def __init__(self, hidden_dim: int, num_heads: int, mlp_ratio: 
        super().__init__()
        self.hidden_dim = hidden_dim
        self.num_heads = num_heads

        # --- LayerNorm (affine=False: 학습 파라미터 없이 정규화만 수
        self.norm1 = nn.LayerNorm(hidden_dim, elementwise_affine=Fa
        self.norm2 = nn.LayerNorm(hidden_dim, elementwise_affine=Fa

        # --- Multi-Head Self-Attention ---
        self.attn = nn.MultiheadAttention(
            embed_dim=hidden_dim,
            num_heads=num_heads,
            batch_first=True,  # (B, T, D) 형태 입력
        )

        # --- Feed-Forward Network (MLP) ---
        mlp_hidden = int(hidden_dim * mlp_ratio)
        self.mlp = nn.Sequential(
            nn.Linear(hidden_dim, mlp_hidden),
            nn.GELU(approximate="tanh"),  # DiT는 GELU 활성화 사용
            nn.Linear(mlp_hidden, hidden_dim),
        )

        # --- adaLN-Zero 변조 파라미터 생성기 ---
        # 조건 벡터 c로부터 6개 벡터를 한 번에 생성:
        #   gamma1, beta1, alpha1 (Attention용)
        #   gamma2, beta2, alpha2 (MLP용)
        self.adaLN_modulation = nn.Sequential(
            nn.SiLU(),  # 활성화 함수
            nn.Linear(hidden_dim, 6 * hidden_dim, bias=True),
        )

        # ★ Zero 초기화: gate(alpha)가 0에서 시작하도록
        # 이렇게 하면 학습 초기에 각 블록이 항등 함수처럼 동작
        nn.init.zeros_(self.adaLN_modulation[-1].weight)
        nn.init.zeros_(self.adaLN_modulation[-1].bias)

    def forward(self, x: torch.Tensor, c: torch.Tensor) -> torch.Te
        """
```

```
        Args:
            x: (B, T, D) — 토큰 시퀀스 (패치 임베딩)
            c: (B, D)    — 조건 벡터 (timestep + class 임베딩)
        Returns:
            (B, T, D) — 변환된 토큰 시퀀스
        """
        # 조건 벡터로부터 6개의 변조 파라미터 생성
        modulation = self.adaLN_modulation(c)  # (B, 6*D)
        gamma1, beta1, alpha1, gamma2, beta2, alpha2 = modulation.ch
        # 각각 (B, D) 형태

        # --- Attention 경로 ---
        # adaLN: LayerNorm 후 scale/shift 적용
        h = self.norm1(x)                                # (B, T, D)
        h = gamma1.unsqueeze(1) * h + beta1.unsqueeze(1)  # 브로드캐
        h, _ = self.attn(h, h, h)                        # Self-Attent
        x = x + alpha1.unsqueeze(1) * h                  # gate 적용 후

        # --- MLP 경로 ---
        h = self.norm2(x)
        h = gamma2.unsqueeze(1) * h + beta2.unsqueeze(1)
        h = self.mlp(h)
        x = x + alpha2.unsqueeze(1) * h                  # gate 적용 후

        return x


# --- 사용 예시 ---
if __name__ == "__main__":
    B, T, D = 4, 256, 1152   # 배치, 토큰 수(256 패치), DiT-XL 차원
    num_heads = 16

    block = AdaLNZeroDiTBlock(hidden_dim=D, num_heads=num_heads)

    x = torch.randn(B, T, D)   # 패치 임베딩
    c = torch.randn(B, D)      # 조건 벡터 (t_emb + y_emb)

    out = block(x, c)
    print(f"입력 형태: {x.shape}")   # torch.Size([4, 256, 1152])
    print(f"출력 형태: {out.shape}")  # torch.Size([4, 256, 1152])
    print(f"파라미터 수: {sum(p.numel() for p in block.parameters())
```

A.8 PyTorch 구현: Patchify / Unpatchify

DiT의 입력/출력 단계에서 이미지 잠재 표현을 패치 토큰으로 변환하고 다시 복원하는 과정이다.

```python
import torch
import torch.nn as nn


class PatchEmbed(nn.Module):
    """
    이미지(잠재 표현)를 패치로 분할하고 임베딩하는 모듈.
    예: 32×32×4 → 2×2 패치 → 256개 토큰, 각 (2*2*4=16)차원 → Linear →
    """
    def __init__(
        self,
        img_size: int = 32,
        patch_size: int = 2,
        in_channels: int = 4,
        embed_dim: int = 1152,
    ):
        super().__init__()
        self.img_size = img_size
        self.patch_size = patch_size
        self.num_patches = (img_size // patch_size) ** 2  # 256

        # Conv2d를 kernel_size=stride=patch_size로 사용하면
        # 패치 분할 + 선형 투영을 한 번에 수행할 수 있다
        self.proj = nn.Conv2d(
            in_channels, embed_dim,
            kernel_size=patch_size, stride=patch_size
        )

        # 학습 가능한 위치 임베딩
        self.pos_embed = nn.Parameter(
            torch.zeros(1, self.num_patches, embed_dim)
        )
        # 정규분포로 초기화 (표준편차 0.02)
        nn.init.normal_(self.pos_embed, std=0.02)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        """
        Args:
            x: (B, C, H, W) — 잠재 표현 (예: B, 4, 32, 32)
        Returns:
            (B, num_patches, embed_dim) — 패치 토큰 시퀀스
        """
        # Patchify + Linear projection
        x = self.proj(x)           # (B, embed_dim, H/p, W/p)
        x = x.flatten(2)           # (B, embed_dim, num_patches)
        x = x.transpose(1, 2)     # (B, num_patches, embed_dim)

        # 위치 임베딩 추가
        x = x + self.pos_embed

        return x
```

```python
class Unpatchify(nn.Module):
    """
    Transformer 출력 토큰을 다시 이미지 형태로 복원하는 모듈.
    DiT의 최종 출력: 노이즈 예측 (epsilon) + 분산 예측 (Sigma)
    따라서 출력 채널 = 2 * in_channels (예: 2*4 = 8)
    """
    def __init__(
        self,
        img_size: int = 32,
        patch_size: int = 2,
        in_channels: int = 4,
        embed_dim: int = 1152,
    ):
        super().__init__()
        self.img_size = img_size
        self.patch_size = patch_size
        self.out_channels = 2 * in_channels  # 노이즈 + 분산
        self.grid_size = img_size // patch_size  # 16

        # 최종 LayerNorm + Linear 투영
        self.norm = nn.LayerNorm(embed_dim, elementwise_affine=Fals
        self.proj = nn.Linear(
            embed_dim,
            patch_size * patch_size * self.out_channels  # 2*2*8 =
        )

        # adaLN 최종 변조 (gamma, beta)
        self.adaLN_final = nn.Sequential(
            nn.SiLU(),
            nn.Linear(embed_dim, 2 * embed_dim, bias=True),
        )
        nn.init.zeros_(self.adaLN_final[-1].weight)
        nn.init.zeros_(self.adaLN_final[-1].bias)

    def forward(self, x: torch.Tensor, c: torch.Tensor) -> torch.Te
        """
        Args:
            x: (B, num_patches, embed_dim) — Transformer 출력
            c: (B, embed_dim) — 조건 벡터
        Returns:
            (B, out_channels, H, W) — 복원된 이미지 형태
        """
        # 최종 adaLN 적용
        gamma, beta = self.adaLN_final(c).chunk(2, dim=-1)
        x = gamma.unsqueeze(1) * self.norm(x) + beta.unsqueeze(1)

        # 선형 투영: embed_dim → patch_size^2 * out_channels
        x = self.proj(x)  # (B, num_patches, p*p*C_out)

        # Reshape: (B, num_patches, p*p*C_out) → (B, C_out, H, W)
        B = x.shape[0]
```

```
        B = x.shape[0]
        p = self.patch_size
        g = self.grid_size
        x = x.reshape(B, g, g, p, p, self.out_channels)
        x = x.permute(0, 5, 1, 3, 2, 4)    # (B, C_out, g, p, g, p)
        x = x.reshape(B, self.out_channels, g * p, g * p)  # (B, C_
```

```
        return x


# --- 사용 예시 ---
if __name__ == "__main__":
    B = 4
    img_size, patch_size, in_ch, D = 32, 2, 4, 1152

    patchify = PatchEmbed(img_size, patch_size, in_ch, D)
    unpatchify = Unpatchify(img_size, patch_size, in_ch, D)

    # 잠재 표현 (VAE 인코더 출력)
    z = torch.randn(B, in_ch, img_size, img_size)
    c = torch.randn(B, D)

    tokens = patchify(z)
    print(f"Patchify: {z.shape} → {tokens.shape}")
    # Patchify: torch.Size([4, 4, 32, 32]) → torch.Size([4, 256, 11

    output = unpatchify(tokens, c)
    print(f"Unpatchify: {tokens.shape} → {output.shape}")
    # Unpatchify: torch.Size([4, 256, 1152]) → torch.Size([4, 8, 32,
    #   8채널 = 4(노이즈 예측) + 4(분산 예측)
```

Part B: Consistency Models — 1~4 스텝 생성

> 📄 Song et al., "Consistency Models", ICML 2023

B.1 동기

디퓨전/Flow Matching은 10~1000 스텝이 필요하다. **1 스텝**으로 줄일 수 없을까?

> 💡 **왜 1-step 생성이 절실한가?** Diffusion 모델의 품질은 뛰어나지만, 실용적 관점에서 심각한 병목이 있다. DDPM은 1000 스텝, DDIM은 50 스텝, 고품질 Stable Diffusion은 20~50 스텝이 필요하다. 각 스텝마다 전체 신경망(수억 파라미터의 U-Net 또는 DiT)을 한 번씩 호출해야 한다. 이는 다음과 같은 실제 문제를 야기한다:
>
> - **실시간 응용 불가**: 비디오 생성(30fps)에서 한 프레임당 50회 신경망 호출이 필요하면, 초당 1500회 호출이 필요하다. 현재 GPU로는 불가능하다.
> - **인터랙티브 편집 불가**: 사용자가 "이 부분을 빨간색으로 바꿔줘"라고 했을 때, 결과를 보는 데 10초씩 걸리면 사용 경험이 나빠진다.

> - **비용 문제**: 클라우드 API에서 이미지 1장 생성에 50회 추론 = 비용 50배. 대규모 서비스에서는 추론 비용이 핵심 경쟁력이다.
>
> 만약 **단 1회**의 신경망 호출로 동일한 품질의 이미지를 생성할 수 있다면, 위의 모든 문제가 해결된다. Consistency Model은 정확히 이 목표를 달성한다.

B.2 핵심 아이디어: 자기 일관성 (Self-Consistency)

ODE 궤적(trajectory) 위의 모든 점은 같은 시작점 x_0에서 왔다:

$$x_0 \xrightarrow{\text{ODE}} x_{t_1} \xrightarrow{\text{ODE}} x_{t_2} \xrightarrow{\text{ODE}} \cdots \xrightarrow{\text{ODE}} x_T$$

일관성 함수 f_θ: 궤적 위의 어떤 점이든 같은 x_0로 매핑:

$$f_\theta(x_{t_1}, t_1) = f_\theta(x_{t_2}, t_2) = \cdots = x_0$$

> 어떤 시점 t에서 시작하든, 한 번의 함수 호출로 x_0를 직접 얻는다.

자기 일관성 — 핵심 통찰을 깊이 이해하기:

기존 Diffusion 모델이 "왜" 여러 스텝이 필요한지부터 생각해 보자. DDPM/DDIM은 벡터장 $v(x_t, t)$ 또는 노이즈 $\epsilon_\theta(x_t, t)$를 예측한다. 이들은 ODE 궤적 위의 **접선 방향 (tangent direction)** 을 알려줄 뿐이다. 접선을 따라 작은 스텝씩 이동해야 궤적을 따라갈 수 있으므로, 많은 스텝이 필요하다. 큰 스텝을 밟으면 궤적에서 이탈하여 품질이 떨어진다.

Consistency Model의 발상 전환은 이렇다: 접선 방향을 예측하는 대신, **궤적의 끝점 (endpoint) 자체를 직접 예측**하자. 궤적 위의 어디에서든 x_0를 직접 알려주는 함수를 학습하면, 단 1회의 함수 호출로 끝난다.

이를 수학적으로 표현하면:

$$f(x_t, t) = f(x_{t'}, t') \quad \text{for all } t, t' \in [0, T] \text{ on the same PF-ODE trajectory}$$

이 성질이 **자기 일관성(self-consistency)** 이다. 같은 궤적 위의 어떤 두 점 (x_t, t)와 $(x_{t'}, t')$를 함수 f에 넣어도 **반드시 같은 값** x_0가 나와야 한다. "일관성"이라는 이름은 여기서 유래한다.

왜 이것이 1-step 생성을 가능하게 하는가? 자기 일관성이 성립하면, $f(x_T, T) = f(x_0, 0) = x_0$이다. 즉, 순수 노이즈 x_T를 넣어도, 깨끗한 데이터 x_0를 넣어도, 같은 궤적 위에 있기만 하면 동일한 출력 x_0을 얻는다. 따라서 $x_T \sim \mathcal{N}(0, I)$를 샘플링한 후 $f(x_T, T)$를 한 번 계산하면 바로 이미지 x_0을 얻는다.

B.3 경계 조건

$f_\theta(x_0, 0) = x_0$ (항등 조건).

> 💡 **왜 경계 조건이 반드시 필요한가?** 일관성 함수 f_θ는 궤적의 어떤 점이든 시작점 x_0로 매핑해야 한다. 그렇다면 시작점 자체인 x_0를 넣었을 때는? 당연히 x_0 그대로 나와야 한다. 즉 $f_\theta(x_0, 0) = x_0$이어야 한다. 이 조건 없이 자유롭게 학습하면, 신경망이 **모든 입력을 같은 평균적인 이미지로 매핑**하는 trivial solution에 빠질 수 있다. 경계 조건은 "노이즈가 0인 깨끗한 데이터는 그대로 보존하라"는 제약을 강제하여, 의미 있는 일관성 함수의 학습을 보장한다.

이를 보장하기 위해:

$$f_\theta(x, t) = \begin{cases} x & \text{if } t = 0 \\ F_\theta(x, t) & \text{if } t > 0 \end{cases}$$

또는 skip connection으로:

$$f_\theta(x, t) = c_{\text{skip}}(t) \cdot x + c_{\text{out}}(t) \cdot F_\theta(x, t)$$

$c_{\text{skip}}(0) = 1, c_{\text{out}}(0) = 0$으로 설정.

Skip connection 형태를 선택하는 이유:

위의 두 가지 형태 중, 실제 구현에서는 skip connection 형태가 사용된다. 첫 번째 형태($t = 0$일 때 분기)는 $t = 0$ 근처에서 불연속이 발생할 수 있어 학습이 불안정하다. Skip connection 형태는 $c_{\text{skip}}(t)$와 $c_{\text{out}}(t)$가 t에 대해 연속 함수이므로, $t \to 0$일 때 자연스럽게 $f_\theta(x, t) \to x$가 된다.

구체적인 계수 설계 (논문의 설정):

$$c_{\text{skip}}(t) = \frac{\sigma_{\text{data}}^2}{t^2 + \sigma_{\text{data}}^2}, \quad c_{\text{out}}(t) = \frac{t \cdot \sigma_{\text{data}}}{\sqrt{t^2 + \sigma_{\text{data}}^2}}$$

여기서 σ_{data}는 데이터의 표준편차 (하이퍼파라미터, 보통 0.5). 이 설계의 의미를 살펴보면:

- $t = 0$: $c_{\text{skip}} = 1, c_{\text{out}} = 0$ → 출력 = x (경계 조건 만족)
- $t \gg \sigma_{\text{data}}$: $c_{\text{skip}} \approx 0, c_{\text{out}} \approx \sigma_{\text{data}}$ → 출력이 거의 신경망 F_θ에 의해 결정됨
- 즉, 노이즈가 작으면 입력을 거의 그대로 쓰고, 노이즈가 크면 신경망의 예측에 의존한다. 이는 직관적으로도 합리적이다.

B.4 Consistency Distillation

사전 학습된 디퓨전 모델이 있을 때, 일관성 모델로 **증류**:

```
1. x₀ ~ p_data
2. t_n, t_{n+1}: 인접 타임스텝
3. x_{t_{n+1}} = forward(x₀, t_{n+1})
4. x_{t_n} = one_step_denoise(x_{t_{n+1}}, t_{n+1} → t_n)  # 디퓨전
5. Loss = ‖f_θ(x_{t_{n+1}}, t_{n+1}) - f_{θ⁻}(x_{t_n}, t_n)‖²
```

- θ^-: EMA (Exponential Moving Average) 파라미터
- 인접 두 점의 출력이 일치하도록 → 궤적 전체가 같은 x_0로 매핑

Consistency Distillation의 단계별 해설:

위 알고리즘의 각 단계를 직관적으로 이해해 보자:

1. **데이터 샘플링** ($x_0 \sim p_{\text{data}}$): 학습 데이터에서 깨끗한 이미지 하나를 가져온다.
2. **인접 타임스텝 선택** (t_n, t_{n+1}): 전체 타임스텝 $0 = t_0 < t_1 < \cdots < t_N = T$에서 인접한 두 시점을 고른다. 예를 들어 $t_n = 5.0, t_{n+1} = 5.5$처럼 가까운 두 시점이다.
3. **Forward process**: x_0에 t_{n+1} 수준의 노이즈를 추가하여 $x_{t_{n+1}}$을 만든다. 이것은 분석적으로 계산 가능하다: $x_{t_{n+1}} = x_0 + t_{n+1} \cdot \epsilon, \epsilon \sim \mathcal{N}(0, I)$.
4. **Teacher 모델로 한 스텝 디노이즈**: **사전 학습된 디퓨전 모델**(teacher)을 사용하여 $x_{t_{n+1}}$에서 x_{t_n}으로 ODE 한 스텝을 밟는다. Teacher가 알려주는 벡터장을 따라 $t_{n+1} \to t_n$으로 이동한 결과가 $\hat{x}_{t_n}$이다.
5. **일관성 손실 계산**: 이제 $(x_{t_{n+1}}, t_{n+1})$과 $(\hat{x}_{t_n}, t_n)$은 **같은 ODE 궤적 위의 인접한 두 점**이다. 따라서 일관성 함수 f_θ에 넣으면 같은 값이 나와야 한다. 이 차이를 줄이는 것이 Loss다.

왜 EMA 파라미터 θ^-를 사용하는가? 두 점 모두에 같은 파라미터 θ를 사용하면, 모든 출력을 상수로 만드는 trivial solution($f_\theta \equiv \text{const}$)으로 붕괴할 수 있다. θ^-는 θ의 지수 이동 평균으로, 느리게 업데이트되는 "안정적 타겟"을 제공한다. 이는 DQN에서 target network를 쓰는 것, BYOL/MoCo에서 momentum encoder를 쓰는 것과 같은 원리다.

B.5 Consistency Training (증류 없이)

디퓨전 모델 없이도 직접 학습 가능:

$$\mathcal{L}_{\text{CT}} = \mathbb{E}\left[d\left(f_\theta(x_{t_{n+1}}, t_{n+1}),\ f_{\theta^-}(x_{t_n}, t_n)\right)\right]$$

여기서 x_{t_n}은 ODE 한 스텝이 아니라 **Euler 한 스텝**으로 근사.

> 💡 **왜 Consistency Training이 더 어렵지만 더 우아한가?** Consistency Distillation은 사전 학습된 디퓨전 모델(teacher)이 필요하다. Teacher를 먼저 학습시키고 → 그 위에서 증류하는 2단계 파이프라인이다. Consistency Training은 teacher 없이 **처음부터** 일관성 함수를 직접 학습한다. Teacher가 알려주는 정확한 ODE 궤적 정보가 없으므로, 대신 Euler 근사($x_{t_n} \approx x_{t_{n+1}} + (t_n - t_{n+1}) \cdot \nabla \log p(x_{t_{n+1}})$)를 사용한다. 이 근사는 t_n과 t_{n+1}이 가까울수록 정확해지므로, 학습 중 타임스텝 간격을 점차 줄여가는 curriculum이 필요하다. 구현이 더 까다롭지만, 별도의 teacher

> 모델 학습이 불필요하므로 전체 파이프라인이 단순해지고, 이론적으로 teacher의 오류가 전파되지 않는다는 장점이 있다.

B.6 생성

```
# 1-step 생성
x_T ~ N(0, I)
x_0 = f_θ(x_T, T)              # 한 번의 함수 호출로 끝!

# Multi-step (품질 향상)
x_T ~ N(0, I)
x_0 = f_θ(x_T, T)              # 1단계 예측
x_{T/2} = forward(x_0, T/2)     # 중간 지점으로 노이즈 추가
x_0 = f_θ(x_{T/2}, T/2)         # 2단계 예측 (정제)
```

1-step vs Multi-step 생성의 직관적 이해:

1-step: 순수 노이즈 x_T에서 곧바로 x_0를 예측한다. 가장 빠르지만, 한 번의 함수 호출로 "노이즈 → 이미지"의 전체 변환을 수행해야 하므로 세밀한 디테일이 부족할 수 있다. 비유하자면, 화가가 빈 캔버스에 **한 번의 붓질로** 완성작을 그리는 것과 같다.

Multi-step: 1-step 결과를 "초안"으로 사용하고, 여기에 중간 수준의 노이즈를 다시 추가한 후 한 번 더 일관성 함수를 호출하여 정제한다. 이 과정을 반복하면 점진적으로 품질이 개선된다. 비유하면, 화가가 초안을 그린 후 → 일부러 약간 흐릿하게 만들고 → 다시 선명하게 그려 디테일을 추가하는 과정이다.

핵심은 Multi-step에서도 **각 스텝이 독립적인 x_0 예측**이라는 것이다. 기존 DDPM/DDIM은 각 스텝이 "작은 디노이즈"여서 한 스텝만으로는 의미 있는 이미지가 나오지 않지만, Consistency Model은 어떤 스텝에서든 완전한 x_0 예측이 가능하다. 따라서 1-step으로도 합리적인 이미지를 생성하면서, 추가 스텝으로 품질을 점진적으로 높일 수 있는 **유연한 속도-품질 트레이드오프**를 제공한다.

B.7 성능

모델	스텝	FID (CIFAR-10)
DDPM	1000	3.17
DDIM	50	4.67
Consistency Model (CD)	**1**	**3.55**
Consistency Model (CD)	**2**	**2.93**
Progressive Distillation	4	3.0

> 1 스텝으로 DDPM 1000 스텝에 근접하는 성능.

성능 표 분석:

이 결과의 의미를 정량적으로 분석해 보자:

- **속도 대비 품질**: Consistency Model(1 step)은 DDPM 대비 **1000배 빠르면서** FID 차이가 0.38에 불과하다 (3.55 vs 3.17). DDIM(50 step)보다 **50배 빠르면서** FID는 오히려 더 좋다 (3.55 vs 4.67).
- **2-step의 효과**: 스텝을 1 → 2로 늘리면 FID가 3.55 → 2.93으로 크게 개선된다. 추론 비용은 2배지만, 이는 DDPM 1000 step보다 여전히 **500배 빠르다**.
- **Progressive Distillation과의 비교**: Progressive Distillation은 4 step에서 FID 3.0을 달성한다. Consistency Model은 2 step만으로 이를 능가한다 (FID 2.93).

실무 관점에서 이것은 혁명적이다. A100 GPU에서 Stable Diffusion이 50 step으로 이미지 1장에 약 5초가 걸린다면, Consistency Model은 1 step에 약 0.1초 — **실시간 인터랙티브 생성**이 가능해지는 수준이다.

B.8 PyTorch 구현: Consistency Model 추론

⚠️ 아래는 학습이 완료된 Consistency Model로 이미지를 생성하는 추론 코드이다. 1-step 및 multi-step 생성 모두 구현한다.

```python
import torch
import torch.nn as nn
import torch.nn.functional as F


class ConsistencyModelWrapper(nn.Module):
    """
    Consistency Model의 추론 래퍼.
    skip connection 구조로 경계 조건 f(x, 0) = x를 보장한다.

    핵심 수식:
      f_theta(x, t) = c_skip(t) * x + c_out(t) * F_theta(x, t)
      여기서 c_skip(0) = 1, c_out(0) = 0
    """
    def __init__(
        self,
        denoiser: nn.Module,
        sigma_data: float = 0.5,
        sigma_min: float = 0.002,
        sigma_max: float = 80.0,
    ):
        super().__init__()
        self.denoiser = denoiser        # 신경망 F_theta (예: U-Net)
        self.sigma_data = sigma_data    # 데이터 표준편차 (하이퍼파라미
        self.sigma_min = sigma_min      # 최소 노이즈 레벨
        self.sigma_max = sigma_max      # 최대 노이즈 레벨

    def c_skip(self, sigma: torch.Tensor) -> torch.Tensor:
        """Skip connection 계수: sigma → 0일 때 1로 수렴"""
        return self.sigma_data ** 2 / (sigma ** 2 + self.sigma_data

    def c_out(self, sigma: torch.Tensor) -> torch.Tensor:
        """Output 스케일링 계수: sigma → 0일 때 0으로 수렴"""
        return sigma * self.sigma_data / (sigma ** 2 + self.sigma_da

    def c_in(self, sigma: torch.Tensor) -> torch.Tensor:
        """Input 스케일링 (수치 안정성)"""
        return 1.0 / (sigma ** 2 + self.sigma_data ** 2).sqrt()

    def forward(self, x: torch.Tensor, sigma: torch.Tensor) -> torch
        """
        일관성 함수: f_theta(x, sigma) → x_0 예측

        Args:
            x: (B, C, H, W) — 노이즈가 포함된 이미지
            sigma: (B,) 또는 스칼라 — 현재 노이즈 레벨
        Returns:
            (B, C, H, W) — 디노이즈된 이미지 (x_0 예측)
        """
        # sigma를 (B, 1, 1, 1) 형태로 브로드캐스트 준비
        if sigma.dim() == 0:
            sigma = sigma.expand(x.shape[0])
```

```
        s = sigma.view(-1, 1, 1, 1)

        # 신경망에 입력 전 스케일링
        scaled_input = self.c_in(s) * x

        # 신경망 F_theta 호출
        F_out = self.denoiser(scaled_input, sigma)

        # Skip connection으로 경계 조건 보장
        return self.c_skip(s) * x + self.c_out(s) * F_out


@torch.no_grad()
def consistency_sample_onestep(
    model: ConsistencyModelWrapper,
    shape: tuple,
    sigma_max: float = 80.0,
    device: str = "cuda",
) -> torch.Tensor:
    """
    1-step 생성: 순수 노이즈에서 한 번의 함수 호출로 이미지 생성.

    Args:
        model: 학습된 Consistency Model
        shape: 생성할 이미지 형태 (B, C, H, W)
        sigma_max: 시작 노이즈 레벨
    Returns:
        (B, C, H, W) — 생성된 이미지
    """
    # Step 1: 가우시안 노이즈 샘플링
    x = torch.randn(shape, device=device) * sigma_max

    # Step 2: 단 한 번의 함수 호출로 x_0 예측
    sigma = torch.full((shape[0],), sigma_max, device=device)
    x_0 = model(x, sigma)

    return x_0


@torch.no_grad()
def consistency_sample_multistep(
    model: ConsistencyModelWrapper,
    shape: tuple,
    sigmas: list[float] = None,
    device: str = "cuda",
) -> torch.Tensor:
    """
    Multi-step 생성: 여러 번의 denoise-add_noise 사이클로 품질 향상.

    원리: 1-step 결과에 중간 수준의 노이즈를 다시 추가한 뒤
          일관성 함수를 다시 호출하여 정제(refine)한다.
```

```
    Args:
        model: 학습된 Consistency Model
        shape: 생성할 이미지 형태 (B, C, H, W)
        sigmas: 노이즈 스케줄 [sigma_max, sigma_mid, ...] (내림차순)
    Returns:
        (B, C, H, W) — 생성된 이미지
    """
    if sigmas is None:
        # 기본: 3-step 스케줄 (논문의 실험 설정 참고)
        sigmas = [80.0, 24.4, 5.84, 0.9]

    # Step 1: 초기 노이즈
    x = torch.randn(shape, device=device) * sigmas[0]

    for i in range(len(sigmas)):
        # 현재 노이즈 레벨에서 x_0 예측
        sigma = torch.full((shape[0],), sigmas[i], device=device)
        x_0 = model(x, sigma)

        # 마지막 스텝이면 종료 (노이즈 재추가 불필요)
        if i < len(sigmas) - 1:
            # x_0에 다음 레벨의 노이즈를 추가하여 정제 루프 계속
            next_sigma = sigmas[i + 1]
            noise = torch.randn_like(x_0)
            x = x_0 + next_sigma * noise

    return x_0


# --- 사용 예시 (가상의 Denoiser 사용) ---
if __name__ == "__main__":
    # 간단한 가상 Denoiser (실제로는 U-Net 등 사용)
    class DummyDenoiser(nn.Module):
        def __init__(self, channels=3):
            super().__init__()
            self.net = nn.Sequential(
                nn.Conv2d(channels, 64, 3, padding=1),
                nn.ReLU(),
                nn.Conv2d(64, channels, 3, padding=1),
            )
        def forward(self, x, sigma):
            return self.net(x)

    device = "cpu"  # 예시용
    denoiser = DummyDenoiser(channels=3)
    model = ConsistencyModelWrapper(denoiser).to(device)

    # 1-step 생성
    img_1step = consistency_sample_onestep(model, (2, 3, 32, 32), de
    print(f"1-step 생성 결과: {img_1step.shape}")  # torch.Size([2,
```

```
# Multi-step 생성 (3단계)
img_multi = consistency_sample_multistep(
    model, (2, 3, 32, 32),
    sigmas=[80.0, 24.4, 5.84],
    device=device,
)
print(f"Multi-step 생성 결과: {img_multi.shape}")  # torch.Size(
```

Part C: Visual Autoregressive Models (VAR)

> 📄 Tian et al., "Visual Autoregressive Modeling: Scalable Image Generation via Next-Scale Prediction", NeurIPS 2024 **Best Paper**

C.1 기존 Visual AR의 한계 (강의 5 DALL-E)

DALL-E 방식: 이미지를 1D 토큰 시퀀스로 펼쳐서 자기회귀:

```
이미지 → VQ-VAE → 래스터 순서: [t₁, t₂, t₃, ..., t₁₀₂₄]
                                 좌→우, 위→아래
```

문제:

- 2D 공간 구조가 1D로 펼쳐져 정보 손실
- LLM에서 관찰되는 **스케일링 법칙이 이미지에서 나타나지 않았음**
- Diffusion 모델에 FID 기준으로 밀림

> 💡 **왜 래스터 순서(raster scan order)가 이미지에 부자연스러운가?** 텍스트에서 자기회귀(autoregressive)는 자연스럽다: "The cat sat on the"까지 읽으면 다음 단어 "mat"을 예측할 수 있다. 언어는 본래 좌에서 우로 읽는 **1D 순차 구조**이기 때문이다. 하지만 이미지는 **2D 공간 구조**다. 래스터 순서로 이미지를 1D로 펼치면, 공간적으로 인접한 픽셀이 시퀀스 상에서는 매우 멀어질 수 있다. 예를 들어 32×32 이미지에서 위치 $(0, 31)$과 $(1, 0)$은 공간적으로 바로 아래에 있지만, 래스터 순서로는 31개 토큰이 떨어져 있다. Transformer가 이 먼 거리의 의존성을 학습하려면 Self-Attention을 통해 명시적으로 연결해야 하는데, 이는 시퀀스가 길어질수록 비효율적이다.
>
> 더 근본적인 문제는, 이 순서가 이미지 생성의 **인과 구조(causal structure)** 와 맞지 않는다는 것이다. 인간 화가는 "왼쪽 위 → 오른쪽 아래"로 그림을 그리지 않는다. 먼저 전체 구도를 잡고 (스케치), 큰 영역의 색을 채우고, 마지막에 세밀한 디테일을 추가한다. 즉 **coarse-to-fine (거친 것에서 세밀한 것으로)** 순서가 이미지 생성의 자연스러운 인과 구조다. VAR은 바로 이 통찰을 활용한다.

C.2 VAR의 핵심: Next-Scale Prediction

토큰 단위가 아니라 해상도(스케일) 단위로 자기회귀:

```
[1×1]      → [2×2]      → [4×4]      → [8×8]      → [16×16]      → [32×32]
(전체구조)  (대형태)     (중형태)     (소형태)     (디테일)       (세밀디테일
 1토큰       4토큰        16토큰       64토큰       256토큰        1024토큰
```

$$p(x) = p(r_1) \cdot p(r_2 \mid r_1) \cdot p(r_3 \mid r_1, r_2) \cdots p(r_K \mid r_1, \ldots, r_{K-1})$$

- r_k: 스케일 k의 토큰 맵 (해상도 $2^k \times 2^k$의 VQ-VAE 코드)
- 이전 스케일들이 모두 조건으로 주어짐

Multi-scale tokenization (다중 스케일 토큰화) 상세 설명:

VAR의 전제는 하나의 이미지를 **여러 해상도로 동시에** VQ-VAE로 인코딩하는 것이다. 같은 이미지를 1×1, 2×2, 4×4, ..., 32×32 해상도에서 각각 VQ-VAE 코드로 변환한다. 각 스케일이 담는 정보는 본질적으로 다르다:

스케일	해상도	토큰 수	담는 정보	비유
r_1	1×1	1	전체 이미지의 평균 색상/밝기	아주 멀리서 본 그림
r_2	2×2	4	상하좌우 4분면의 대략적 구성	눈을 가늘게 뜨고 본 그림
r_3	4×4	16	주요 객체의 위치와 대략적 형태	썸네일
r_4	8×8	64	객체의 윤곽선, 색상 분포	저해상도 사진
r_5	16×16	256	텍스처, 세부 형태	중해상도 사진
r_6	32×32	1024	미세한 텍스처, 엣지	고해상도 사진

자기회귀의 방향이 핵심이다: $r_1 \to r_2 \to \cdots \to r_K$ 순서로 생성한다. 먼저 전체 구도(r_1)를 결정한 후, 그 위에 점점 세밀한 정보($r_2, r_3, \ldots$)를 추가한다. 이것은 인간의 그림 그리기 과정과 동일하다: 러프 스케치 → 형태 잡기 → 색칠 → 디테일.

Causal attention mask (인과적 어텐션 마스크) 구조:

VAR에서 Transformer의 Self-Attention 마스크는 "나중 스케일은 이전 스케일을 볼 수 있지만, 이전 스케일은 나중 스케일을 볼 수 없다"는 규칙을 따른다. 5개 스케일의 경우 마스크 구조는 다음과 같다:

```
          r₁    r₂    r₃    r₄    r₅
r₁  [   O     X     X     X     X   ]    ← r₁은 자기 자신만 볼 수 있음
r₂  [   O     O     X     X     X   ]    ← r₂는 r₁과 자신을 볼 수 있음
r₃  [   O     O     O     X     X   ]    ← r₃는 r₁, r₂, 자신을 볼 수
r₄  [   O     O     O     O     X   ]    ← r₄는 r₁~r₃과 자신을 볼 수
r₅  [   O     O     O     O     O   ]    ← r₅는 모든 스케일을 볼 수 있

O = attend 가능 (볼 수 있음),  X = attend 불가 (마스킹됨)
```

이 마스크의 핵심: **정보는 coarse에서 fine으로만 흐른다**. 고해상도 스케일은 저해상도 스케일의 정보를 참고하여 "어떤 세부 사항을 그릴지" 결정하지만, 저해상도 스케일은 아직 존재하지 않는 고해상도 정보를 참고할 수 없다. 이것이 자기회귀의 인과성(causality)을 보장한다. 같은 스케일 내의 토큰들은 서로를 볼 수 있는데, 이는 같은 스케일의 토큰들이 **병렬로 동시에** 생성되기 때문이다.

C.3 구조

```
[Class Token + 1×1 토큰]
        ↓
[Transformer Block × N]
        ↓
[Predict 2×2 토큰 맵 (4개 토큰을 병렬로!)]
        ↓
[Class Token + 1×1 + 2×2 토큰]
        ↓
[Transformer Block × N]
        ↓
[Predict 4×4 토큰 맵 (16개 토큰을 병렬로!)]
        ↓
  ... 반복 ...
```

핵심: 각 스케일 내의 토큰들은 **병렬로** 생성 (독립 가정). 스케일 간만 자기회귀.

> 💡 **왜 스케일 내 병렬 생성이 가능한가?** 기존 next-token AR(GPT, DALL-E)에서는 각 토큰이 이전 토큰에 의존하므로 순차 생성이 필수다. VAR은 핵심적인 가정을 도입한다: **같은 스케일 내의 토큰들은 조건부 독립(conditionally independent)이다.** 즉, 이전 스케일들 $r_1, \ldots, r_{k-1}$이 주어지면, 현재 스케일 r_k의 각 토큰은 다른 토큰과 독립적으로 생성할 수 있다. 수식으로 표현하면:
>
> $$p(r_k \mid r_1, \ldots, r_{k-1}) = \prod_{i=1}^{|r_k|} p(r_k^{(i)} \mid r_1, \ldots, r_{k-1})$$
>
> 이 가정이 정확히 성립하지는 않지만 (같은 스케일 내 토큰 간에도 상관관계가 있다), 실험적으로 충분히 좋은 근사임이 입증되었다. 이 가정 덕분에 $16 \times 16 = 256$개 토큰을 **한 번에** 병렬로 예측할 수 있어 생성 속도가 획기적으로 빨라진다.

생성 속도 비교:

- **DALL-E (next-token)**: 256개 토큰을 순차적으로 → 256번의 Transformer forward pass
- **VAR (next-scale, 스케일 10개)**: 10번의 forward pass (각 스케일에서 1번)만으로 전체 이미지 생성
- 속도 차이: 약 **25배** 빠름 (단순 forward pass 횟수 기준)

C.4 기존 토큰별 AR vs VAR 비교

	토큰별 AR (DALL-E)	VAR
자기회귀 단위	토큰 1개	**스케일 전체**
생성 순서	좌→우, 위→아래	**저해상도 → 고해상도**
병렬성	❌ 순차적	✅ **스케일 내 병렬**
공간 구조	1D로 펼침 (손실)	**2D 구조 보존**
스케일링 법칙	❌ 관찰 안 됨	✅ **LLM처럼 관찰됨**
FID (ImageNet 256)	~5+	**1.73**

> 💡 **왜 VAR에서는 스케일링 법칙이 나타나고, 기존 Visual AR에서는 나타나지 않았을까?** 이 질문은 VAR 논문의 핵심 기여 중 하나다. 논문의 가설은 다음과 같다: 기존 next-token AR은 래스터 순서로 이미지를 생성하기 때문에, 모델이 풀어야 하는 문제가 **이미지의 본질적 구조와 불일치**한다. 모델을 키워도 "부자연스러운 순서로 예측하기"라는 잘못된 문제 설정 자체가 병목이 된다. 반면 VAR의 coarse-to-fine 순서는 이미지의 자연스러운 계층 구조(hierarchy)와 일치하므로, 모델 용량이 커지면 각 스케일의 예측이 정확해지고, 이것이 전체 품질 향상으로 직결된다. 올바른 문제 설정 위에서 스케일링해야 스케일링 법칙이 나타나는 것이다.

C.5 스케일링 법칙의 발견

VAR에서 처음으로 이미지 생성에서 **LLM과 유사한 스케일링 법칙**이 관찰되었다:

모델 크기	파라미터	FID↓	IS↑
VAR-d16	310M	3.30	274.4
VAR-d20	600M	2.57	302.6
VAR-d24	1.0B	2.09	312.9
VAR-d30	**2.0B**	**1.73**	**350.2**

모델을 키울수록 **일관되게** 성능이 개선된다. Diffusion에서는 이러한 깔끔한 스케일링이 관찰되지 않았다.

스케일링 결과 심층 분석:

- **파라미터 2배 → FID 개선 예측 가능**: 310M → 600M (약 2배) 에서 FID 3.30 → 2.57 (22% 개선), 600M → 1.0B (약 1.7배) 에서 2.57 → 2.09 (19% 개선), 1.0B → 2.0B (2배) 에서 2.09 → 1.73 (17% 개선). 개선 폭이 점차 줄어들지만 **일관되게 개선**된다는 것이 핵심이다.
- **Inception Score(IS)도 동시에 개선**: FID(다양성+품질)와 IS(품질)가 모두 개선되므로, mode collapse 없이 진정한 품질 향상이 일어나고 있음을 확인할 수 있다.
- **DiT와의 비교**: DiT-XL/2는 675M 파라미터에 FID 2.27을 달성한다. VAR-d24는 비슷한 1.0B 파라미터에 FID 2.09로, **더 적은 추론 스텝**(10 vs 250)으로 더 나은 FID를 달성한다. VAR-d30(2.0B)은 FID 1.73으로, 논문 발표 시점 기준 ImageNet 256x256 class-conditional 생성의 state-of-the-art였다.
- **왜 Diffusion에서는 이런 스케일링이 안 되나?** Diffusion 모델의 성능은 모델 크기뿐 아니라 노이즈 스케줄, 샘플링 스텝 수, classifier-free guidance 강도 등 여러 하이퍼파라미터에 민감하게 의존한다. 모델을 키울 때 이 하이퍼파라미터들도 함께 재조정해야 하므로, "파라미터 수만의 함수"로 성능을 예측하기 어렵다.

C.6 MAR: VQ 없는 Visual AR

> 📄 Li et al. (Kaiming He 그룹), "Autoregressive Image Generation without Vector Quantization", NeurIPS 2024

VQ-VAE의 이산 토큰 → **연속 토큰**으로 확장:

- 각 토큰의 분포를 **Diffusion Loss**로 모델링
- Cross-Entropy 대신 per-token diffusion 사용
- Masked Autoregressive: 랜덤 순서로 토큰을 마스킹/예측

> DALL-E(강의 5)의 "이산 토큰 + Cross-Entropy" 패러다임에서 벗어나, 연속 공간에서 AR + Diffusion을 결합한 최신 접근.

> 💡 **왜 VQ(벡터 양자화)를 제거하려 하는가?** VQ-VAE는 이미지를 이산 토큰으로 변환하여 언어 모델 기법을 적용할 수 있게 해주지만, 근본적인 한계가 있다. 양자화(quantization)는 **정보 손실**을 유발한다 -- 연속 벡터를 가장 가까운 코드북 벡터로 "반올림"하는 것이므로, 원본과 양자화된 표현 사이에 반드시 오차가 발생한다. 코드북 크기를 키우면 오차를 줄일 수 있지만, 그러면 Cross-Entropy 손실의 클래스 수가 늘어나 학습이 어려워진다. MAR은 이 딜레마를 해결한다: **양자화를 아예 하지 않고**, 각 토큰의 연속 분포를 소규모 Diffusion 모델로 직접 모델링한다. 이를 "Diffusion Loss"라 부르며, Transformer는 각 위치의 조건 벡터만 출력하고, 그 조건 하에서 토큰 값의 확률 분포를 작은 MLP 기반 diffusion이 학습한다.

Masked Autoregressive의 핵심 아이디어: MAR은 기존의 좌→우 순차 생성 대신, **랜덤 순서**로 토큰을 생성한다. 학습 시 토큰의 일부를 랜덤하게 마스킹하고, 마스킹되지 않은 토큰을 조건으로 마스킹된 토큰을 예측한다. 생성 시에는 모든 토큰이 마스킹된 상태에서 시작하여, 여러 라운드에 걸쳐 점진적으로 마스크를 벗겨간다. 이 방식은 BERT의 마스크 언어 모델링과 유사하지만, 생성(generation)에 사용된다는 점이 다르다.

C.7 PyTorch 구현: VAR Next-Scale Prediction (간소화)

⚠️ 아래는 VAR의 핵심 아이디어인 "스케일별 자기회귀 생성"을 간소화하여 구현한 코드이다. 실제 VAR 논문의 전체 구현보다 단순화되어 있지만, next-scale prediction의 핵심 흐름을 이해하는 데 충분하다.

```python
import torch
import torch.nn as nn
import torch.nn.functional as F
import math


class VARTransformerBlock(nn.Module):
    """VAR에서 사용하는 Transformer 블록 (간소화)"""
    def __init__(self, dim: int, num_heads: int, mlp_ratio: float =
        super().__init__()
        self.norm1 = nn.LayerNorm(dim)
        self.attn = nn.MultiheadAttention(dim, num_heads, batch_firs
        self.norm2 = nn.LayerNorm(dim)
        self.mlp = nn.Sequential(
            nn.Linear(dim, int(dim * mlp_ratio)),
            nn.GELU(),
            nn.Linear(int(dim * mlp_ratio), dim),
        )

    def forward(self, x: torch.Tensor, attn_mask: torch.Tensor = Noı
        # Pre-norm + Self-Attention + 잔차 연결
        h = self.norm1(x)
        h, _ = self.attn(h, h, h, attn_mask=attn_mask)
        x = x + h
        # Pre-norm + MLP + 잔차 연결
        x = x + self.mlp(self.norm2(x))
        return x


class VARNextScalePredictor(nn.Module):
    """
    VAR: Next-Scale Prediction 모델 (간소화 버전).

    스케일 시퀀스 예시 (ImageNet 256):
      [1x1] → [2x2] → [4x4] → [8x8] → [16x16]
      1개     4개     16개     64개     256개 토큰

    각 스케일에서 이전 스케일들의 모든 토큰을 조건으로 받아
    현재 스케일의 토큰 맵을 병렬로 예측한다.
    """
    def __init__(
        self,
        codebook_size: int = 4096,   # VQ-VAE 코드북 크기
        embed_dim: int = 1024,       # Transformer 차원
        num_heads: int = 16,
        depth: int = 16,             # Transformer 블록 수
        num_classes: int = 1000,     # 조건 클래스 수
        scale_schedule: tuple = (1, 2, 4, 8, 16),  # 각 스케일의 해상
    ):
        super().__init__()
        self.codebook_size = codebook_size
        self.embed_dim = embed_dim
```

```python
        self.scale_schedule = scale_schedule
        self.num_scales = len(scale_schedule)

        # 코드북 토큰 임베딩 (VQ-VAE 코드 인덱스 → 벡터)
        self.tok_embed = nn.Embedding(codebook_size, embed_dim)

        # 클래스 조건 임베딩
        self.cls_embed = nn.Embedding(num_classes, embed_dim)

        # 스케일별 위치 임베딩 (각 스케일마다 별도)
        self.pos_embeds = nn.ParameterList()
        for res in scale_schedule:
            num_tokens = res * res
            self.pos_embeds.append(
                nn.Parameter(torch.zeros(1, num_tokens, embed_dim))
            )
        # 초기화
        for pe in self.pos_embeds:
            nn.init.normal_(pe, std=0.02)

        # Transformer 블록
        self.blocks = nn.ModuleList([
            VARTransformerBlock(embed_dim, num_heads)
            for _ in range(depth)
        ])

        # 최종 출력 헤드: 각 위치에서 코드북 인덱스에 대한 확률 분포
        self.head_norm = nn.LayerNorm(embed_dim)
        self.head = nn.Linear(embed_dim, codebook_size, bias=False)

    def _build_causal_mask(self, total_tokens: int, scale_boundarie
        """
        스케일별 인과적 마스크 생성.
        - 이전 스케일의 토큰은 볼 수 있음
        - 같은 스케일 내 토큰은 서로 볼 수 있음 (병렬 생성)
        - 미래 스케일의 토큰은 볼 수 없음
        """
        mask = torch.ones(total_tokens, total_tokens, device=device,
        # True = 마스킹(attend 불가)

        for i, (start_i, end_i) in enumerate(
            zip(scale_boundaries[:-1], scale_boundaries[1:])
        ):
            for j, (start_j, end_j) in enumerate(
                zip(scale_boundaries[:-1], scale_boundaries[1:])
            ):
                if j <= i:
                    # 현재 또는 이전 스케일 → attend 허용
                    mask[start_i:end_i, start_j:end_j] = False

        return mask
```

```python
@torch.no_grad()
def generate(self, class_labels: torch.Tensor, temperature: flo
    """
    스케일별 자기회귀 생성.

    Args:
        class_labels: (B,) — 클래스 인덱스
        temperature: 샘플링 온도 (높을수록 다양성 증가)
    Returns:
        각 스케일의 토큰 인덱스 리스트 [(B, 1, 1), (B, 2, 2), ...,
    """
    B = class_labels.shape[0]
    device = class_labels.device
    generated_scales = []

    # 조건: 클래스 토큰 (시퀀스의 맨 앞에 추가)
    cls_token = self.cls_embed(class_labels).unsqueeze(1)  # (B,

    # 현재까지 생성된 모든 토큰을 축적
    all_tokens = cls_token  # 시작: 클래스 토큰만

    for scale_idx, res in enumerate(self.scale_schedule):
        num_new_tokens = res * res

        # 현재 스케일의 위치에 "빈" 쿼리 토큰 추가
        # (학습된 위치 임베딩으로 초기화)
        query_tokens = self.pos_embeds[scale_idx].expand(B, -1,
        input_seq = torch.cat([all_tokens, query_tokens], dim=1

        # Transformer 블록 통과
        h = input_seq
        for block in self.blocks:
            h = block(h)

        # 마지막 num_new_tokens개 위치에서 logits 추출
        logits = self.head(self.head_norm(h[:, -num_new_tokens:,
        # (B, num_new_tokens, codebook_size)

        # 온도 적용 후 Categorical 샘플링
        probs = F.softmax(logits / temperature, dim=-1)
        sampled_indices = torch.multinomial(
            probs.view(B * num_new_tokens, -1), num_samples=1
        ).view(B, num_new_tokens)
        # (B, num_new_tokens)

        # 결과 저장 (2D 맵 형태로 reshape)
        generated_scales.append(sampled_indices.view(B, res, res

        # 생성된 토큰을 임베딩하여 다음 스케일 입력에 추가
        new_embeds = self.tok_embed(sampled_indices)  # (B, num_
        new_embeds = new_embeds + self.pos_embeds[scale_idx].exp
```

```
            all_tokens = torch.cat([all_tokens, new_embeds], dim=1)

        return generated_scales


# --- 사용 예시 ---
if __name__ == "__main__":
    device = "cpu"  # 예시용

    # 간소화된 VAR 모델 생성
    model = VARNextScalePredictor(
        codebook_size=4096,
        embed_dim=512,       # 실제는 1024~2048
        num_heads=8,
        depth=8,             # 실제는 16~30
        num_classes=1000,
        scale_schedule=(1, 2, 4, 8, 16),  # 5개 스케일
    ).to(device)

    print(f"VAR 모델 파라미터: {sum(p.numel() for p in model.paramete

    # 클래스 조건 생성
    class_labels = torch.tensor([207, 360], device=device)  # 골든 
    scales = model.generate(class_labels, temperature=1.0)

    print("\n스케일별 생성 결과:")
    for i, s in enumerate(scales):
        res = model.scale_schedule[i]
        print(f"  스케일 {i+1}: {res}x{res} → 토큰 맵 형태 {s.shape}"
        # 스케일 1: 1x1 → torch.Size([2, 1, 1])
        # 스케일 2: 2x2 → torch.Size([2, 2, 2])
        # ...
        # 스케일 5: 16x16 → torch.Size([2, 16, 16])

    total_tokens = sum(r * r for r in model.scale_schedule)
    print(f"\n총 토큰 수: {total_tokens}")
    # 1 + 4 + 16 + 64 + 256 = 341 토큰
    # (기존 AR은 256개 토큰을 순차 생성, VAR는 5 스텝만에 완료)
```

Part D: 하이브리드 모델 — AR + Flow Matching

D.1 FlowAR (2024)

VAR의 스케일별 자기회귀 + Flow Matching을 결합:

```
[1×1 스케일] → Flow Matching으로 생성
     ↓ (조건)
[2×2 스케일] → Flow Matching으로 생성
     ↓ (조건)
  ... 반복 ...
```

각 스케일의 토큰 분포를 Cross-Entropy가 아닌 **Flow Matching**으로 모델링.

> 💡 **왜 Cross-Entropy 대신 Flow Matching을 사용하는가?** VAR은 각 스케일의 토큰을 VQ-VAE 코드북 인덱스로 표현하고 Cross-Entropy로 학습한다. 이는 코드북 크기에 의존하며, 양자화로 인한 정보 손실이 발생한다. FlowAR은 각 스케일의 **연속 벡터 분포**를 Flow Matching으로 직접 모델링한다. 이렇게 하면 VQ-VAE의 양자화 병목을 완전히 제거하면서, VAR의 coarse-to-fine 구조의 장점은 유지할 수 있다. 즉, VAR의 "스케일별 자기회귀"라는 매크로 구조와 Flow Matching의 "연속 분포 모델링"이라는 마이크로 구조를 결합한 것이다.

D.2 패러다임 수렴

2024-2025년의 핵심 트렌드: **Autoregressive + Diffusion/Flow의 경계가 허물어지고 있다.**

```
[Autoregressive]                    [Diffusion]
  DALL-E (2021)                      DDPM (2020)
  VQ-GAN+Transformer                LDM / SD (2022)
       ↓                                ↓
  VAR (2024)                         DiT (2023)
  MAR (2024)                         Flow Matching (2023)
       ↓                                ↓
       └──────────→ 융합 ←──────────────┘
                 FlowAR, MAR+Diffusion Loss
                 "어떤 방식으로든 분포를 잘 학습하면 된다"
```

패러다임 수렴의 의미를 깊이 생각해 보자:

2020~2022년에는 Autoregressive 모델과 Diffusion 모델이 **완전히 별개의 패러다임**으로 취급되었다. AR은 이산 토큰 + Cross-Entropy + 순차 생성이고, Diffusion은 연속 공간 + MSE/Score Matching + 반복 디노이즈였다. 두 접근법은 수학적 프레임워크부터 구현까지 거의 모든 면에서 달랐다.

하지만 2024~2025년의 연구들은 이 경계가 인위적인 것임을 보여주고 있다:

- **MAR**: AR 프레임워크에서 Diffusion Loss를 사용 (AR + Diffusion)
- **FlowAR**: AR의 스케일별 구조에서 Flow Matching을 사용 (AR + Flow)
- **Consistency Model**: Diffusion의 ODE 궤적을 1-step 매핑으로 변환 (Diffusion의 AR화)
- **DiT**: Diffusion의 백본을 Transformer로 교체 (LLM 아키텍처의 Diffusion 적용)

이 수렴이 시사하는 바는 명확하다: **중요한 것은 특정 패러다임이 아니라, 고차원 데이터의 확률 분포를 정확하게 학습하는 능력**이다. AR이든 Diffusion이든, 그것은 분포를 학습하는 서로 다른 "경로"일 뿐이다. 각 경로의 장점(AR의 스케일링 법칙, Diffusion의 연속 공간 모델링, Flow Matching의 효율적 학습)을 결합하는 것이 현재 최전선의 연구 방향이다.

핵심 수식 요약

DiT

$$\mathrm{adaLN}(h, c) = \gamma(c) \odot \mathrm{LN}(h) + \beta(c), \quad c = \mathrm{MLP}(t_{\mathrm{emb}} + y_{\mathrm{emb}})$$

해석: 조건 벡터 c (타임스텝 + 클래스 임베딩)로부터 scale γ와 shift β를 생성하여, LayerNorm의 출력을 조건에 따라 변조한다. 모든 레이어의 모든 토큰이 강제로 조건의 영향을 받는다.

$$\mathrm{output} = h + \alpha(c) \odot \mathrm{Block}(\mathrm{adaLN}(h, c)), \quad \alpha \text{ zero-initialized}$$

해석: gate α가 0에서 시작하여, 학습 초기에 각 블록이 항등 함수로 동작하게 한다. 이것이 "Zero" 초기화의 핵심이다.

Consistency Model

$$f_\theta(x_{t_1}, t_1) = f_\theta(x_{t_2}, t_2) \quad \forall t_1, t_2 \text{ on same ODE trajectory}$$

해석: 같은 ODE 궤적 위의 어떤 두 점을 넣어도 같은 출력 x_0가 나온다. 이 성질이 1-step 생성을 가능하게 한다: $f_\theta(x_T, T) = x_0$.

$$f_\theta(x, t) = c_{\mathrm{skip}}(t) \cdot x + c_{\mathrm{out}}(t) \cdot F_\theta(x, t), \quad c_{\mathrm{skip}}(0) = 1,\ c_{\mathrm{out}}(0) = 0$$

해석: Skip connection으로 경계 조건 $f_\theta(x_0, 0) = x_0$을 구조적으로 보장한다.

$$\mathcal{L}_{\mathrm{CD}} = \mathbb{E}\left[\| f_\theta(x_{t_{n+1}}, t_{n+1}) - f_{\theta^-}(x_{t_n}, t_n) \|^2\right]$$

해석: 인접한 두 점의 일관성 함수 출력 차이를 최소화한다. θ^-는 EMA 파라미터로, trivial solution 방지를 위한 안정적 타겟이다.

VAR

$$p(x) = \prod_{k=1}^{K} p(r_k \mid r_1, \ldots, r_{k-1})$$

해석: 이미지의 확률을 스케일별 조건부 확률의 곱으로 분해한다. r_k는 스케일 k의 토큰 맵이다. 기존 next-token AR과 달리 각 스케일 내 토큰은 병렬 생성되므로, K번(보통 10)의 forward pass만으로 전체 이미지를 생성한다.

세 모델의 핵심 기여 한 줄 요약:

모델	핵심 기여	핵심 수식/개념
DiT	이미지 생성에 스케일링 법칙을 가져옴	adaLN-Zero 조건 주입
Consistency Model	1-step 고품질 생성을 가능하게 함	Self-consistency on ODE trajectories
VAR	이미지에 자연스러운 coarse-to-fine AR을 도입	Next-scale prediction

다음 강의 미리보기

다음 강의에서는 생성 모델의 **응용**을 다룬다:

- **비디오 생성**: Sora, Movie Gen (시간 축 확장)
- **3D 생성**: Score Distillation Sampling
- **Physical AI**: Diffusion Policy, World Models

이 강의에서 배운 DiT, Consistency Model, VAR은 모두 이러한 응용의 **기반 기술**이다. Sora는 DiT 아키텍처를 비디오로 확장한 것이고, 실시간 비디오 생성에는 Consistency Model의 소수 스텝 생성이 필수적이며, 3D/비디오의 계층적 생성에는 VAR의 coarse-to-fine 사고방식이 자연스럽게 적용된다.

참고문헌 (References)

DiT 관련

1. **Peebles, W., & Xie, S.** (2023). *Scalable Diffusion Models with Transformers.* In Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV). arXiv:2212.09748
2. **Esser, P., Kulal, S., Blattmann, A., et al.** (2024). *Scaling Rectified Flow Transformers for High-Resolution Image Synthesis* (Stable Diffusion 3). In International Conference on Machine Learning (ICML). arXiv:2403.03206
3. **Vaswani, A., et al.** (2017). *Attention Is All You Need.* Advances in Neural Information Processing Systems (NeurIPS). arXiv:1706.03762

Consistency Models 관련

4. **Song, Y., Dhariwal, P., Chen, M., & Sutskever, I.** (2023). *Consistency Models.* In International Conference on Machine Learning (ICML).

arXiv:2303.01469

5. **Song, Y., & Dhariwal, P.** (2024). *Improved Techniques for Training Consistency Models.* In International Conference on Learning Representations (ICLR). arXiv:2310.14189

6. **Salimans, T., & Ho, J.** (2022). *Progressive Distillation for Fast Sampling of Diffusion Models.* In International Conference on Learning Representations (ICLR). arXiv:2202.00512

VAR / Visual Autoregressive 관련

7. **Tian, K., Jiang, Y., Yuan, Z., Peng, B., & Wang, L.** (2024). *Visual Autoregressive Modeling: Scalable Image Generation via Next-Scale Prediction.* In Advances in Neural Information Processing Systems (NeurIPS). **Best Paper Award.** arXiv:2404.02905

8. **Li, T., Tian, Y., Li, H., Deng, M., & He, K.** (2024). *Autoregressive Image Generation without Vector Quantization.* In Advances in Neural Information Processing Systems (NeurIPS). arXiv:2406.11838

9. **Ramesh, A., Pavlov, M., Goh, G., et al.** (2021). *Zero-Shot Text-to-Image Generation* (DALL-E). In International Conference on Machine Learning (ICML). arXiv:2102.12092

배경 / 기초

10. **Ho, J., Jain, A., & Abbeel, P.** (2020). *Denoising Diffusion Probabilistic Models.* Advances in Neural Information Processing Systems (NeurIPS). arXiv:2006.11239

11. **Rombach, R., Blattmann, A., Lorenz, D., Esser, P., & Ommer, B.** (2022). *High-Resolution Image Synthesis with Latent Diffusion Models.* In IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). arXiv:2112.10752

12. **Lipman, Y., Chen, R. T. Q., Ben-Hamu, H., Nickel, M., & Le, M.** (2023). *Flow Matching for Generative Modeling.* In International Conference on Learning Representations (ICLR). arXiv:2210.02747